ЛР4. FP32 проти INT8

Дедлайн: до початку лекції 13. **Оцінювання:** 10 балів. **Стек:** Python 3.12, PyTorch, torchvision, ONNX, ONNX Runtime, NumPy, scikit-learn, Pillow. Усі вимірювання виконуються на CPU вашого ноутбука; GPU й окрема плата не потрібні.

Мета роботи

Перевести модель із ЛР3 у формат ONNX, побудувати з неї восьмибітну версію статичною квантизацією після навчання (post-training quantization) і відповісти на питання: **чи дає квантизація після навчання виграш у затримці на вашому CPU і якою зміною якості та розміру файлу він купується?**

Оцінюється протокол вимірювання, а не конкретне прискорення. Якщо на вашому процесорі INT8 не швидша за FP32 або навіть повільніша, такий результат, коректно виміряний і пояснений, не знижує оцінку.

Стартового коду немає. Модель, дані, калібрувальна вибірка, параметри експорту й квантизації та протокол вимірювання нижче зафіксовано однозначно, а для поділу, калібрувальної вибірки й експорту наведено контрольні числа. Розбіжність із ними означає помилку реалізації. Числа, що залежать від моделі (якість, діапазони ваг), наведено з еталонного розв'язку як орієнтир; для вашої моделі вони будуть іншими.

Вихідні дані та фіксовані налаштування

Модель

Модель береться з ЛР3. Це файл `lab3_model.pt`, словник із ключами `state_dict`, `regime`, `seed`, `classes`, `img_size`. Завантажуйте його через `torch.load(path, map_location="cpu", weights_only=True)`, будуйте `torchvision.models.resnet18(weights=None)`, замінюйте `model.fc` на `torch.nn.Linear(512, 6)`, завантажуйте `state_dict` і переводьте модель у `eval()`. Порядок класів у `classes` має збігатися з ЛР3: `buildings`, `forest`, `glacier`, `mountain`, `sea`, `street`.

Дані й поділ

Набір той самий, що в ЛР3, Intel Image Classification (посилання). Поділ береться з вашого `split.csv` із ЛР3; validation у цій роботі не використовується. Перед роботою перевірте дві частини поділу за кількістю зображень і сумою числових імен файлів (`20057.jpg` → 20057):

Частина	Кількість	Сума імен
train	1200	11 993 084
test	1502	33 351 444

Це рядок «Разом» контрольної таблиці ЛР3. Розбіжність треба виправити в ЛР3 до початку ЛР4.

Перетворення для всіх зображень (і калібрувальних, і тестових) таке саме, як для test у ЛР3: `Resize((160, 160))` → `ToTensor()` → `Normalize(mean, std)` зі статистиками ImageNet $\text{mean} = (0{,}485;\ 0{,}456;\ 0{,}406)$, $\text{std} = (0{,}229;\ 0{,}224;\ 0{,}225)$. Розмір входу береться з `img_size` у файлі моделі. Зображення частини train лежать у `seg_train/seg_train/<клас>/`, частини test у `seg_test/seg_test/<клас>/`; відкривайте їх через Pillow з `.convert("RGB")`. Порядок тестових зображень у всіх обчисленнях такий самий, як порядок рядків test у `split.csv`.

Фіксовані налаштування

Етап	Налаштування
Експорт	`torch.onnx.export` з `dynamo=False`, `opset_version=17`, вхід `input`, вихід `logits`, змінний розмір пакета (`dynamic_axes` за віссю 0); модель у `eval()`
Підготовка до квантизації	`quant_pre_process(fp32, prep, skip_symbolic_shape=True)`
Квантизація	`quantize_static` з `quant_format=QuantFormat.QDQ`, `activation_type=QuantType.QUInt8`, `weight_type=QuantType.QInt8`, `per_channel=True`, `calibrate_method=CalibrationMethod.MinMax`; решта аргументів за замовчуванням
Сесія	`onnxruntime.InferenceSession` з `providers=["CPUExecutionProvider"]`, `intra_op_num_threads=1`, `inter_op_num_threads=1`, рівень оптимізації графа за замовчуванням
Якість	усі 1502 зображення test, пакети по 64, порядок як у `split.csv`
Затримка	розмір пакета $B = 1$, 1 потік, 3 раунди, у кожному 20 прогрівальних викликів і 100 вимірів на варіант

---

1. Експорт і перевірка відповідності

Експортуйте модель у `lab4_fp32.onnx` і переконайтеся, що файл ONNX обчислює ту саму функцію, що й модель PyTorch. Для всіх 1502 зображень test обчисліть логіти моделі PyTorch (під `torch.no_grad()`) і логіти сесії ONNX Runtime та наведіть:

- максимальну абсолютну різницю логітів $\max |z_{\text{ONNX}} - z_{\text{PyTorch}}|$;
- частку зображень, для яких збігається передбачений клас (argmax);
- macro-F1 на test для PyTorch і для ONNX FP32;
- кількість вузлів `BatchNormalization` і `Conv` в експортованому графі.

Контроль. Різниця логітів порядку 10^{-5} (в еталоні $5{,}25 \cdot 10^{-5}$); вона має бути меншою за 10^{-3}. Збіг класів 1,0000. macro-F1 PyTorch і ONNX FP32 однакові й дорівнюють вашому числу на test у ЛР3 (в еталоні 0,8892). У графі 20 вузлів `Conv` і жодного `BatchNormalization`, бо під час експорту в режимі `eval()` пакетну нормалізацію злито з попередньою згорткою. Розбіжність у класах чи в macro-F1 найчастіше спричиняють інше перетворення входу, модель не в `eval()` або інший порядок класів.

---

2. Калібрування й квантизація

Калібрувальна вибірка (calibration set) складається з 200 зображень частини **train** поділу ЛР3:

1. Візьміть 1200 рядків `split.csv` із `part == "train"` і відсортуйте їх за іменем файлу як рядки.
2. `idx = np.random.default_rng(0).permutation(1200)[:200]`; калібрувальні рядки мають номери `idx` у відсортованому списку кроку 1.

Контроль. 200 зображень, сума числових імен файлів **1 921 470**; за класами `buildings` 40, `forest` 35, `glacier` 36, `mountain` 33, `sea` 29, `street` 27.

Діапазони активацій, оцінені калібруванням, стають частиною моделі, тому test для калібрування не використовується (правило з лекції 8).

Підготуйте граф (`quant_pre_process`) і побудуйте `lab4_int8.onnx` статичною квантизацією з параметрами з таблиці фіксованих налаштувань. Калібрувальні зображення подавайте через `CalibrationDataReader` по одному ($1 \times 3 \times 160 \times 160$). Час експорту й квантизації наведіть у звіті окремо.

---

3. Якість

Для варіантів FP32 і INT8 на всіх 1502 зображеннях test обчисліть macro-F1, частку правильних відповідей (accuracy) і згоду передбачень (prediction agreement) з FP32, тобто частку зображень, для яких варіант передбачає той самий клас, що й FP32.

Порівняння з урахуванням шуму. Обидва варіанти оцінюються на тих самих зображеннях, тому порівнювати їх треба попарно. Нехай b дорівнює кількості зображень, на яких INT8 відповідає правильно, а FP32 помиляється, а c кількості зображень із протилежною ситуацією. Якщо варіанти однаково точні, $b \sim \mathrm{Bin}(b + c,\ 1/2)$, звідки

$$\Delta_{\mathrm{acc}} = \mathrm{acc}_{\mathrm{INT8}} - \mathrm{acc}_{\mathrm{FP32}} = \frac{b - c}{n}, \qquad \mathrm{SE}(\Delta_{\mathrm{acc}}) \approx \frac{\sqrt{b + c}}{n}, \qquad n = 1502.$$

Правило інтерпретації. Якщо $|b - c| \le 2\sqrt{b + c}$, різниця якості **не відрізняється від шуму**. Якщо $|b - c| > 2\sqrt{b + c}$, вона відрізняється від шуму, і її напрям треба назвати.

Варіант	macro-F1	accuracy	Згода з FP32	b	c	$\Delta_{\mathrm{acc}} \pm \mathrm{SE}$
FP32			1	0	0	
INT8						

Порівняйте $\mathrm{SE}(\Delta_{\mathrm{acc}})$ зі стандартною похибкою $\sqrt{p(1-p)/n}$ частки правильних відповідей однієї моделі з ЛР3 і поясніть, чому парна похибка менша в кілька разів.

Орієнтир з еталонного розв'язку: INT8 дає macro-F1 0,8900, згоду 0,9920, $b = 6$, $c = 5$, $\Delta_{\mathrm{acc}} = +0{,}0007 \pm 0{,}0022$. Ці числа залежать від моделі, процесора й версії ONNX Runtime і критерієм оцінювання не є. Згода нижче 0,9 найчастіше означає помилку калібрування (не те перетворення входу, не ті зображення) або експорт моделі не в режимі `eval()`.

---

4. Дослід із потензорними вагами

Поканальна квантизація (per-channel) бере для кожного вихідного каналу c власний діапазон α_c, потензорна (per-tensor) один діапазон $\max_c \alpha_c$ на весь тензор. Перехід до потензорних ваг коштує каналу c приблизно $\log_2(\max_{c'} \alpha_{c'}/\alpha_c)$ біта (лекція 9).

Діапазони каналів. Для кожного вузла `Conv` графа `lab4_fp32.onnx` обчисліть $\alpha_c = \max|w|$ за всіма елементами вихідного каналу c і відношення $r = \max_c \alpha_c / \operatorname{median}_c \alpha_c$. Ваги вузла беруться з `graph.initializer` (`onnx.numpy_helper.to_array`) за іменем другого входу вузла, форма $(C_{\text{out}}, C_{\text{in}}, k_H, k_W)$. Наведіть три згортки з найбільшим r і медіану r за всіма згортками; медіанному каналу потензорний крок коштує $\log_2 r$ біта.

Орієнтир з еталонного розв'язку. Найбільше $r = 6{,}67$ у `/conv1/Conv`, далі $5{,}90$ у `/layer3/layer3.1/conv2/Conv` і $4{,}92$ у `/layer3/layer3.0/downsample/downsample.0/Conv`; медіана за 20 згортками $3{,}44$, тобто $1{,}78$ біта. У першій згортці MobileNetV3 з лекції 9 це відношення сягає 90.

Прогноз до запуску. За цими числами запишіть у README, чи впаде згода з FP32 і чи зміниться macro-F1 понад шум за правилом пункту 3, якщо квантувати ваги потензорно (`per_channel=False`), а все інше лишити без змін. Прогноз не виправляється після запуску.

Запуск. Побудуйте `lab4_int8_per_tensor.onnx` з `per_channel=False` на тій самій калібрувальній вибірці й з рештою тих самих параметрів. Обчисліть для нього рядок таблиці пункту 3 (затримку міряти не потрібно) і зіставте з прогнозом. Поясніть, чому згода передбачень реагує на квантизацію чутливіше, ніж macro-F1, і чому розходження можуть взаємно компенсуватися в $b - c$.

---

5. Затримка

Затримка вимірюється для `lab4_fp32.onnx` і `lab4_int8.onnx` за таким протоколом:

1. Обидві сесії створюються заздалегідь; створення сесії, експорт і квантизація в час не входять.
2. Вхід один і той самий, перше зображення test у порядку `split.csv`, масив `float32` форми $1 \times 3 \times 160 \times 160$, підготовлений до вимірювання.
3. Три раунди. У кожному раунді спершу FP32, потім INT8; для кожного варіанта 20 прогрівальних викликів без запису, потім 100 вимірів.
4. Вимір містить один виклик `sess.run` між двома зчитуваннями `time.perf_counter()`, і нічого більше.
5. Для кожного варіанта p50 і p95 рахуються за всіма 300 вимірами; окремо наводиться p50 кожного раунду.

Під час вимірювання ноутбук підключений до живлення, важкі програми закриті.

Варіант	Файл, МБ	p50, мс	p95, мс	p50 за раундами, мс
FP32				
INT8				

Розмір файлу рахуйте як кількість байтів, поділену на 10^6. Прискорення визначається як відношення p50: $p50_{\text{FP32}}/p50_{\text{INT8}}$.

Правило інтерпретації. Зміну затримки вважайте виміряною, якщо відрізки $[\min, \max]$ значень p50 за раундами для двох варіантів не перетинаються.

Разом із таблицею наведіть модель процесора, операційну систему, версії Python, PyTorch, ONNX, ONNX Runtime, кількість потоків і розмір пакета.

Орієнтир на Apple M3 Pro з ONNX Runtime 1.29.0: FP32 44,71 МБ, p50 21,91 мс, p95 22,09 мс; INT8 11,30 МБ, p50 5,60 мс, p95 5,65 мс; прискорення 3,91. На іншому CPU, особливо на x86 без інструкцій VNNI, прискорення може бути значно меншим або зовсім відсутнім, а якість INT8 може впасти через насичення в цілочисельному множенні. Такий результат, описаний у README, теж коректний.

---

6. Висновок

Завершіть звіт відповіддю на питання роботи:

- у скільки разів змінилися розмір файлу, p50 і p95, і чи виміряна зміна затримки за правилом пункту 5;
- чи відрізняється зміна якості від шуму за правилом пункту 3 і яка згода передбачень;
- чому прискорення на вашому процесорі менше, більше чи дорівнює чотирикратному зменшенню байтів ваг; спирайтеся на модель даху з лекції 9 і на те, чи має ваш CPU цілочисельні ядра для INT8;
- чи придатна INT8-модель для цієї задачі на вашому ноутбуці.

Кожне твердження підкріпіть числами з таблиць пунктів 3–5. Висновок стосується конкретного процесора й конкретної версії ONNX Runtime.

Що здати

Теку `lab4/` у тому самому GitHub-репозиторії, де лежать попередні роботи. Вона містить:

- код у `.py`-файлах, файл залежностей із зафіксованими версіями й одну команду запуску через `uv` з параметрами шляхів до моделі, `split.csv` і локальної копії набору;
- `lab4_results.json`, записаний цією командою: усі числа пунктів 1–5 (зокрема всі 300 вимірів затримки кожного варіанта або їхні p50 і p95 за раундами) і параметри середовища з пункту 5; структура файлу на ваш розсуд;
- `README` (md/pdf/docx) з таблицями й результатами пунктів 1–5, прогнозом і результатом досліду, параметрами середовища й висновком.

Файли `.onnx` і набір до репозиторію не додавайте. Модель береться з теки `lab3/` вашого репозиторію або з GitHub Release, як у ЛР3. Вимушене відхилення від фіксованих налаштувань (наприклад, інша версія бібліотеки зі зміненими аргументами) назвіть у README; незадеклароване відхилення вважається помилкою реалізації.

Що саме має відтворюватися

На машині викладача команда запускається з тією самою моделлю.

- Точно мають збігтися контроль поділу, калібрувальна вибірка (кількість, сума імен і розподіл за класами), результати перевірки експорту з точністю до молодших розрядів різниці логітів, відношення r для згорток і розмір файлу FP32.
- Якість INT8 і числа досліду мають збігтися на процесорі тієї самої архітектури з тією самою версією ONNX Runtime. На іншій архітектурі вони можуть відрізнятися на кілька зображень, тому в README наведіть числа з власного `lab4_results.json`.
- Затримка на машині викладача буде іншою, і збігу не вимагається. Перевіряється, що код реалізує протокол пункту 5 і що числа в README збігаються з вашим `lab4_results.json`.

Оцінювання та захист

Складова	**Бали**
Експорт і перевірка відповідності, калібрування й квантизація за умовою з контролем калібрувальної вибірки	2
Якість: таблиця, парне порівняння й вердикт за правилом	2
Затримка: протокол вимірювання, таблиця, параметри середовища	3
Дослід із потензорними вагами: діапазони каналів, прогноз до запуску, зіставлення	1
Висновок: зміна ресурсів, якість відносно шуму, пояснення прискорення	1
Відтворюваність і повнота звіту	1
Разом	**10**

На захисті студент пояснює власний код і числа. Потрібно вміти пояснити: чому калібрувальна вибірка береться з train; що станеться з порівнянням, якщо FP32 і INT8 виміряти з різною кількістю потоків або без прогріву; чому звітуються p50 і p95 замість середнього; чому парна похибка $\sqrt{b + c}/n$ менша за похибку однієї моделі; скільки бітів коштує потензорний крок медіанному каналу вашої моделі; чому прискорення INT8 залежить від процесора.

Використання зовнішнього коду та генеративних інструментів дозволене із зазначенням джерел у `README`; воно не скасовує вимоги розуміти поданий розв'язок. Для повного зарахування результат має відтворюватися за наведеною командою на машині викладача.